

Recurrent Neural Networks

LSTM and Gated RNNs

Introduction to Deep Learning · Class 28

Outline

Long-Range Dependencies

Why a Simple Cell Forgets

Paths Through Time

The LSTM

Other Gated Cells

Optimising Through Time

Putting It Together

Long-Range Dependencies

Where we are coming from

Classes 26–27: the recurrent cell at work

one cell, one state $\mathbf{h}_t$ carried step to step

the gradient across a lag is a product, one factor per step

an encoder reads the source into one vector $\mathbf{c} = \mathbf{h}_n^e$

attention lets the decoder look back at every state

what we measured

a reader for any length, trained by BPTT

it fell by 40× over 15 steps at initialisation

exact reversal fell from 100% to 21% by length 10

a fix for the decoder — not for the cell itself

- Both measurements say the same thing about the same object: the state of a simple cell does not **hold a fact** across many steps, and the gradient that would teach it to does not **reach** across many steps either
- Class 27 used the cell g as a black box and worked around it. Today the box is opened: why $\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t)$ forgets, and what one change inside it does about it

Classes 26–27; Jurafsky & Martin (2026), §14.5; Goodfellow et al. (2016), §10.7, §10.10.

The problem: a fact that must survive a long wait

- The running sentence again: *The flights the airline was canceling **were** full.* To choose *were*, the network must still know, six words later, that the subject was plural, while *airline was* — singular — passes through in between
- The same shape everywhere: a quotation mark closed after a clause; a name at the top of a paragraph and *she* at the bottom; a source read in full before its translation (Class 27)
- What the state must do: **keep** one thing unchanged for many steps, **ignore** the intervening input where it is concerned, **release** it when the moment comes — and, in training, receive a gradient from that moment back to the arrival

Keeping a fact and using the state for everything else are two jobs. A simple cell has one vector, rewritten at every step, for both.

Jurafsky & Martin (2026), §14.5, example (14.19); Goodfellow et al. (2016), §10.7.

Why the simple cell is not enough

- Picture the state as a **whiteboard**. At every step the simple cell wipes the whole board and rewrites it from what was there and what just came in: $\mathbf{h}_t = \tanh(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t)$. Nothing is copied; everything is recomputed through $\mathbf{U}$ and squashed
- So *flights* survives to *were* only if $\mathbf{U}$ happens to redraw it faithfully five times in a row, through five $\tanh$'s, while also drawing everything else. Class 26 measured what that does to a signal: it shrinks by a factor at each step
- The gradient has the same problem in reverse: it travels the same wire back, through the same $\mathbf{U}$ and the same derivatives, once per step — Class 26's corollary, and the first section of today: it vanishes or explodes
- Making $\mathbf{U}$ bigger does not help: a signal that does not shrink explodes instead. The same matrix cannot do both jobs — keep one thing, transform the rest — for every fact and every step

Class 26, Corollary 5; Goodfellow et al. (2016), §10.7; Bengio et al. (1994).

The idea: a separate memory, with gates

- Give the cell a second vector, a **notebook** $\mathbf{c}_t$, not rewritten but **copied** from step to step, $\mathbf{c}_t = \mathbf{c}_{t-1} + \dots$: no matrix, no **tanh**. A fact written there stays; a gradient sent back along it arrives intact
- Then three learned **switches**, set at every step from the current word and state: an **eraser** (what to wipe), a **pen** (what to write), a **lens** (what to read into $\mathbf{h}_t$ now). Each is a sigmoid, a number in $(0, 1)$ per coordinate multiplied into what it controls: a soft **gate**, learned by back-propagation like everything else
- On the sentence: the pen writes *plural* at *flights*; the eraser keeps it through *the airline was canceling*; the lens reads it at *were*. That is the LSTM: a copied cell and three gates. The GRU: two gates, no separate notebook

Hochreiter & Schmidhuber (1997); Gers et al. (2000); Jurafsky & Martin (2026), §14.5; Goodfellow et al. (2016), §10.10.

Notation for today

$\mathbf{h}_t, \mathbf{c}_t$	hidden state and cell (context) state	$\odot$	element-wise product
$\mathbf{f}_t, \mathbf{i}_t, \mathbf{o}_t$	forget, add (input) and output gates	σ	the logistic sigmoid
$\mathbf{g}_t$	the candidate written into the cell	λ_i	eigenvalues of $\mathbf{U}$, as in Class 13
$\mathbf{u}_t, \mathbf{r}_t$	the GRU's update and reset gates	$\mathbf{J}_t$	$\partial \mathbf{h}_t / \partial \mathbf{h}_{t-1}$, the step Jacobian

- Jurafsky's letters throughout: $\mathbf{U}$ recurrent, $\mathbf{W}$ input, each gate with its own pair. Goodfellow swaps $\mathbf{U}$ and $\mathbf{W}$, writes the cell $\mathbf{s}$, the input gate $\mathbf{g}$ and the output gate $\mathbf{q}$

Jurafsky & Martin (2026), Eqs. 14.20–14.27; Goodfellow et al. (2016), Eqs. 10.40–10.47.

Why a Simple Cell Forgets

The linear case: a power iteration

Theorem 1 (Vanishing and exploding, without a nonlinearity)

For the recurrence $\mathbf{h}_t = \mathbf{U}\mathbf{h}_{t-1}$ with $\mathbf{U} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$ orthogonally diagonalisable,

$$\mathbf{h}_t = \mathbf{U}^t \mathbf{h}_0 = \mathbf{Q}\mathbf{\Lambda}^t \mathbf{Q}^\top \mathbf{h}_0,$$

so the component of $\mathbf{h}_0$ along eigenvector i is multiplied by λ_i^t : geometric decay if $|\lambda_i| < 1$, explosion if $|\lambda_i| > 1$; only the largest survives.

- **Proof.** $(\mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top)^t = \mathbf{Q}\mathbf{\Lambda}^t \mathbf{Q}^\top$ since $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$, and $\mathbf{\Lambda}^t$ has entries λ_i^t . □
- The power method of Class 13, on the recurrent matrix instead of the Hessian. A feedforward net multiplies *different* matrices, which can be scaled to compensate; a recurrent net multiplies the *same* one.

Goodfellow et al. (2016), §10.7, Eqs. 10.36–10.39; Class 13.

The linear case: a power iteration

Theorem 1 (Vanishing and exploding, without a nonlinearity)

For the recurrence $\mathbf{h}_t = \mathbf{U}\mathbf{h}_{t-1}$ with $\mathbf{U} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$ orthogonally diagonalisable,

$$\mathbf{h}_t = \mathbf{U}^t \mathbf{h}_0 = \mathbf{Q}\mathbf{\Lambda}^t \mathbf{Q}^\top \mathbf{h}_0,$$

so the component of $\mathbf{h}_0$ along eigenvector i is multiplied by λ_i^t : geometric decay if $|\lambda_i| < 1$, explosion if $|\lambda_i| > 1$; only the largest survives.

- Proof. $(\mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top)^t = \mathbf{Q}\mathbf{\Lambda}^t\mathbf{Q}^\top$ since $\mathbf{Q}^\top\mathbf{Q} = \mathbf{I}$, and $\mathbf{\Lambda}^t$ has entries λ_i^t . □
- The power method of Class 13, on the recurrent matrix instead of the Hessian. A feedforward net multiplies *different* matrices, which can be scaled to compensate; a recurrent net multiplies the *same* one.

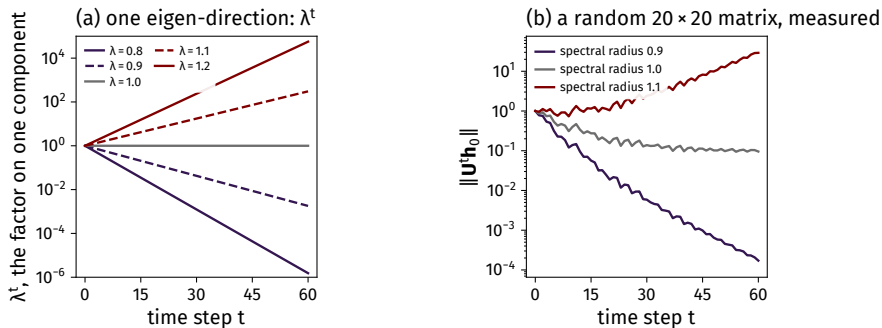
Goodfellow et al. (2016), §10.7, Eqs. 10.36–10.39; Class 13.

The power method: the experiment

- **What is being tested.** Theorem 1 says a repeated linear map multiplies each eigen-component by λ_i^t : after t steps a component with $|\lambda_i| < 1$ is gone and one with $|\lambda_i| > 1$ has taken over. This is the whiteboard being redrawn t times by the same matrix
- **The setup.** (a) Nothing to run: the factor λ^t for five values of λ , on a log scale. (b) A random 20×20 matrix $\mathbf{U}$, rescaled so that its largest eigenvalue in magnitude — its **spectral radius** — is 0.9, 1.0 or 1.1; a random unit vector $\mathbf{h}_0$; multiply 60 times and record $\|\mathbf{U}^t \mathbf{h}_0\|$ at each step
- **What to expect.** At radius 0.9 every component shrinks, so the norm should fall like 0.9^t once the largest component dominates: to about $0.9^{60} \approx 2 \times 10^{-3}$ or below. At 1.1 it should grow like $1.1^t \approx 300$. At 1.0 the largest component holds while the rest die, so the norm should settle rather than fall to zero
- **Why it matters for the cell.** $\mathbf{U}$ is the one matrix the state passes through at every step; without the **tanh**, this is exactly what the state does to a fact

Goodfellow et al. (2016), §10.7, Eqs. 10.36–10.39; Class 13.

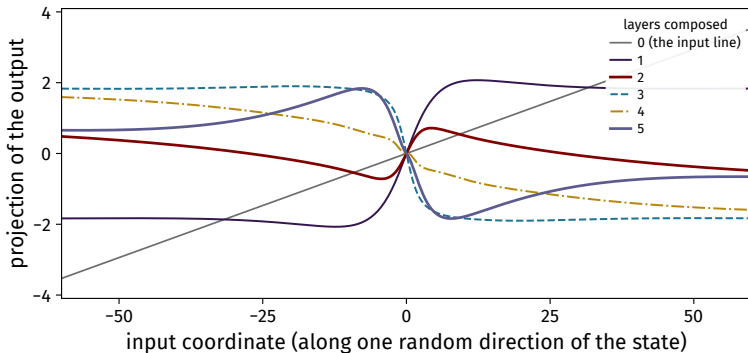
The power method: the result



(a) The theorem, one eigen-direction at a time: λ^t on a log scale is a straight line, down for $|\lambda| < 1$ and up for $|\lambda| > 1$; at 60 steps the two sides differ by ten orders of magnitude. (b) The random matrix, measured: at spectral radius 0.9 the norm is 1.7×10^{-4} after 60 steps, at 1.1 it is 29, at 1.0 it settles near 0.1 — the largest component survives while the others decay. The wobble early on is the components that have not died yet; the slope afterwards is the largest eigenvalue's.

Goodfellow et al. (2016), Eqs. 10.36–10.39.

The composition, drawn



Now with the $\tanh$, recomputed in the layout of Goodfellow's Fig. 10.15: a random linear- $\tanh$ layer applied 1 to 5 times to a 100-dimensional state, seen along one random line through the state (the input coordinate) and projected to one number. The line itself is the grey ramp; after one layer it is already an S; after five it is flat almost everywhere — a tiny derivative — and steep in a few places, with more alternations each time. This is the surface the gradient must cross, once per step, backwards.

Layout of Goodfellow et al. (2016), Fig. 10.15, §10.7.

With the nonlinearity: a bound on the gradient

Proposition 2 (Sufficient condition for vanishing)

For $\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t)$ with $|g'| \leq \gamma$ everywhere ($\gamma = 1$ for $\tanh$, $\frac{1}{4}$ for the sigmoid),

$$\left\| \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \right\| = \left\| \prod_{s=k+1}^t \mathbf{J}_s \right\| \leq (\gamma \|\mathbf{U}\|)^{t-k},$$

so if $\|\mathbf{U}\| < 1/\gamma$ the gradient from t back to k vanishes geometrically in the lag.

- **Proof.** $\mathbf{J}_s = \text{diag}(g') \mathbf{U}$ by the chain rule of Class 26, so $\|\mathbf{J}_s\| \leq \gamma \|\mathbf{U}\|$; the norm of a product is at most the product of the norms. □
- Sufficient, not necessary: it says when the gradient *must* vanish. Either way a distant cause gets an exponentially smaller gradient than a recent one.

Pascanu, Mikolov & Bengio (2013); Goodfellow et al. (2016), §10.7; Bengio et al. (1994).

With the nonlinearity: a bound on the gradient

Proposition 2 (Sufficient condition for vanishing)

For $\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t)$ with $|g'| \leq \gamma$ everywhere ($\gamma = 1$ for $\tanh$, $\frac{1}{4}$ for the sigmoid),

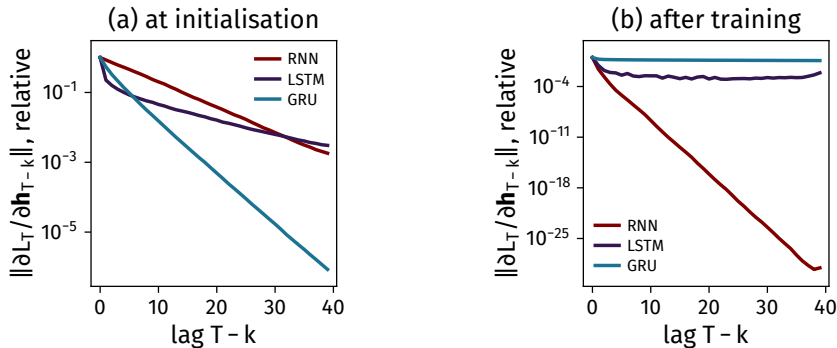
$$\left\| \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \right\| = \left\| \prod_{s=k+1}^t \mathbf{J}_s \right\| \leq (\gamma \|\mathbf{U}\|)^{t-k},$$

so if $\|\mathbf{U}\| < 1/\gamma$ the gradient from t back to k vanishes geometrically in the lag.

- Proof. $\mathbf{J}_s = \text{diag}(g') \mathbf{U}$ by the chain rule of Class 26, so $\|\mathbf{J}_s\| \leq \gamma \|\mathbf{U}\|$; the norm of a product is at most the product of the norms. $\square$
- Sufficient, not necessary: it says when the gradient *must* vanish. Either way a distant cause gets an exponentially smaller gradient than a recent one.

Pascanu, Mikolov & Bengio (2013); Goodfellow et al. (2016), §10.7; Bengio et al. (1994).

The gradient by lag, measured



The corollary of Class 26, measured on three cells — the two gated ones are the next sections' — of width 32, trained here to recall the first symbol of a sequence at its last step: the running sentence's shape, *flights* to *were*, at lag 40. $\|\partial L_T / \partial \mathbf{h}_{T-k}\|$ against the lag k , read off the tape. (a) At initialisation all three fall with the lag. (b) After training the simple RNN's gradient at the first step is 10^{-30} of its value at the last — nothing from *were* reaches *flights* — while the LSTM's is 10^{-2} and the GRU's 0.4: the gated cells have learned a path along which it survives. Goodfellow et al. (2016), §10.7; Jurafsky & Martin (2026), §14.5.

Paths Through Time

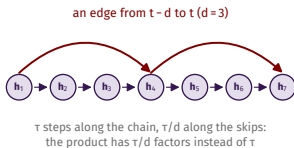
Three ways to a path the gradient survives

- The two sections so far say: the only path from *flights* to *were* runs through $\mathbf{U}$ and a $\tanh$ at every step, and along it the gradient's factors multiply to nothing. Every fix is the same fix: build a **second path** whose factors stay near one
- Three ways to build it, from the books, in order of how much they leave to the network: **skip connections** shorten the path (fewer factors); a **leaky unit** adds a wire with a fixed factor near one; a **gate** adds the same wire and lets the network set the factor at every step
- There is also a way to give up: **echo state networks** fix $\mathbf{U}$ at a chosen spectral radius, never train it, and learn only the read-out — a convex problem, and no gradient through time at all

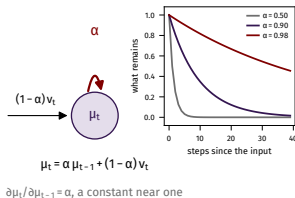
Goodfellow et al. (2016), §10.8–10.10; Lin et al. (1996); Mozer (1992); Jaeger & Haas (2004).

The three paths, drawn

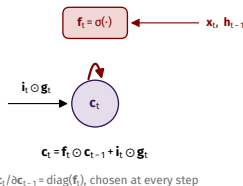
(a) skip connections across time



(b) a leaky unit: a fixed self-loop



(c) a gate: the self-loop's weight is decided by the input



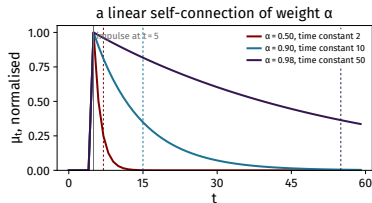
(a) Skip connections across time: an extra edge from h_{t-d} to h_t , so a fact and its gradient cross τ steps in τ/d hops — fewer factors in the product, but d is fixed by hand. (b) A leaky unit: a linear self-loop of weight α , so $\partial \mu_t / \partial \mu_{t-1} = \alpha$ exactly, a constant near one; the inset shows how much of an input remains after t steps for three α — one fixed time scale per unit. (c) A gate: the same self-loop, but its weight f_t is a sigmoid the network computes from x_t and h_{t-1} at every step: keep this fact now, drop it later. The LSTM's cell line is exactly (c). Goodfellow et al. (2016), §10.8, §10.9.2, §10.10; Lin et al. (1996); Mozer (1992).

The leaky unit is a running average

Proposition 3 (Time constant of a leaky unit)

The unit $\mu_t = \alpha\mu_{t-1} + (1 - \alpha)v_t$ unrolls to $\mu_t = (1 - \alpha) \sum_{s \leq t} \alpha^{t-s} v_s$: geometric weights with time constant $1/(1 - \alpha)$ steps.

- **Proof. Substitute the recurrence into itself; the weights sum to one.** $\square$
- Along the self-connection $\partial\mu_t/\partial\mu_{t-1} = \alpha$, a constant near one: the path the gradient survives.
- A gate replaces the constant α by a number in $(0, 1)$ computed at each step.



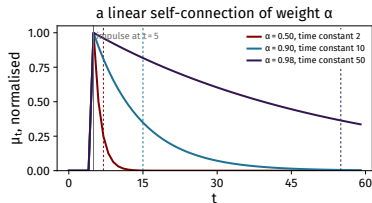
Goodfellow et al. (2016), §10.9.2; Mozer (1992).

The leaky unit is a running average

Proposition 3 (Time constant of a leaky unit)

The unit $\mu_t = \alpha\mu_{t-1} + (1 - \alpha)v_t$ unrolls to $\mu_t = (1 - \alpha) \sum_{s \leq t} \alpha^{t-s} v_s$: geometric weights with time constant $1/(1 - \alpha)$ steps.

- Proof. Substitute the recurrence into itself; the weights sum to one. $\square$
- Along the self-connection $\partial\mu_t/\partial\mu_{t-1} = \alpha$, a constant near one: the path the gradient survives.
- A gate replaces the constant α by a number in $(0, 1)$ computed at each step.



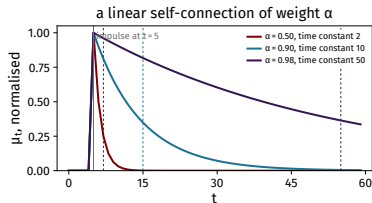
Goodfellow et al. (2016), §10.9.2; Mozer (1992).

The leaky unit is a running average

Proposition 3 (Time constant of a leaky unit)

The unit $\mu_t = \alpha\mu_{t-1} + (1 - \alpha)v_t$ unrolls to $\mu_t = (1 - \alpha) \sum_{s \leq t} \alpha^{t-s} v_s$: geometric weights with time constant $1/(1 - \alpha)$ steps.

- Proof. Substitute the recurrence into itself; the weights sum to one. $\square$
- Along the self-connection $\partial\mu_t/\partial\mu_{t-1} = \alpha$, a constant near one: the path the gradient survives.
- A gate replaces the constant α by a number in $(0, 1)$ computed at each step.



Goodfellow et al. (2016), §10.9.2; Mozer (1992).

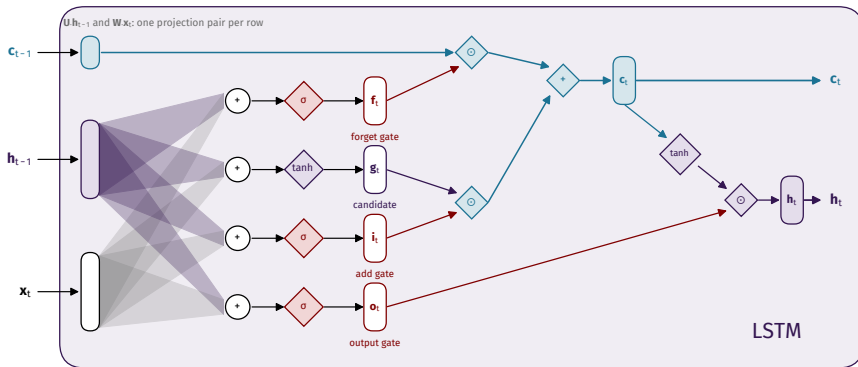
The LSTM

What the LSTM has to do

- Take the notebook idea and make it a cell. Two vectors: the **cell** $\mathbf{c}_t$, which carries context along a copied line, and the **hidden state** $\mathbf{h}_t$, which serves the current output as before. Two jobs, two vectors
- Manage the cell explicitly with three gates, each the same design: a feedforward layer of $\mathbf{x}_t$ and $\mathbf{h}_{t-1}$, a sigmoid, and a pointwise product with the thing gated. The **forget gate** is the eraser, the **add gate** the pen, the **output gate** the lens
- The cell's own update is additive, $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + (\text{what is written})$: the leaky unit of the last section with α replaced by a gate the network sets
- First the picture, then the equations, then why it works

Jurafsky & Martin (2026), §14.5; Hochreiter & Schmidhuber (1997); Gers et al. (2000).

The LSTM cell, drawn



One LSTM unit as a computation graph. In from the left: c_{t-1} , h_{t-1} , x_t . The shaded fans are the projections $U \cdot h_{t-1}$ and $W \cdot x_t$, one pair per row, summed and squashed into the forget gate f_t , the candidate g_t , the add gate i_t and the output gate o_t . Along the top, the cell line (teal): c_{t-1} times f_t , plus $i_t \odot g_t$, is c_t — no weight matrix on that path. h_t is $\tanh(c_t)$ let through by o_t . Out on the right: c_t and h_t . Layout of Jurafsky & Martin (2026), Fig. 14.13.

The LSTM cell, in equations

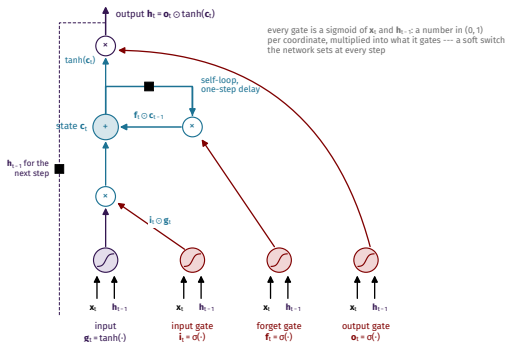
Definition 4 (LSTM)

$\mathbf{f}_t = \sigma(\mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{W}_f \mathbf{x}_t)$	<i>forget gate</i>	$\mathbf{k}_t = \mathbf{c}_{t-1} \odot \mathbf{f}_t$	<i>what survives</i>
$\mathbf{g}_t = \tanh(\mathbf{U}_g \mathbf{h}_{t-1} + \mathbf{W}_g \mathbf{x}_t)$	<i>candidate</i>	$\mathbf{i}_t = \sigma(\mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{W}_i \mathbf{x}_t)$	<i>add gate</i>
$\mathbf{j}_t = \mathbf{g}_t \odot \mathbf{i}_t$	<i>what is added</i>	$\mathbf{c}_t = \mathbf{j}_t + \mathbf{k}_t$	<i>new cell</i>
$\mathbf{o}_t = \sigma(\mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{W}_o \mathbf{x}_t)$	<i>output gate</i>	$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$	<i>new hidden</i>

- The picture, line by line: the eraser $\mathbf{f}_t$ decides what of $\mathbf{c}_{t-1}$ survives; the pen writes $\mathbf{g}_t$, as much as $\mathbf{i}_t$ allows; the lens $\mathbf{o}_t$ decides what is read into $\mathbf{h}_t$. Four weight pairs instead of one; a forget bias of 1 makes the cell start by remembering

Jurafsky & Martin (2026), Eqs. 14.20–14.27; Goodfellow et al. (2016), Eqs. 10.40–10.44.

The LSTM cell, as a block diagram



Goodfellow's view of the same cell, in the layout of his Fig. 10.16. Along the bottom, four units, each fed by x_t and h_{t-1} : the input unit (a $\tanh$, the candidate) and three sigmoid gates. The input is multiplied by the input gate and added into the state; the state has a linear self-loop through a one-step delay (black square) whose weight is the forget gate; the state, through a $\tanh$, is multiplied by the output gate to give the output, which is next step's h_{t-1} (dashed). A gate is a number in $(0, 1)$ per coordinate, multiplied into what it gates: a soft switch the network sets at every step. Layout of Goodfellow et al. (2016), Fig. 10.16, §10.10.1.

Why the cell remembers

Proposition 5 (The cell's gradient is a product of forget gates)

Holding the gates fixed, along the cell's own path $\partial \mathbf{c}_t / \partial \mathbf{c}_k = \prod_{s=k+1}^t \text{diag}(\mathbf{f}_s)$, a product of numbers in $(0, 1)$ with no weight matrix and no activation derivative in it: with $\mathbf{f}_s \approx \mathbf{1}$ the gradient passes unattenuated.

- **Proof.** $\mathbf{c}_s = \mathbf{f}_s \odot \mathbf{c}_{s-1} + \mathbf{j}_s$ is linear in $\mathbf{c}_{s-1}$ with coefficient $\mathbf{f}_s$; chain the steps. □
- Against Proposition 2: the simple cell's factor $\text{diag}(g') \mathbf{U}$ is typically well below one; the LSTM's is a gate the network can hold at one — the **constant error carousel**. On the sentence: with $\mathbf{f}_s \approx \mathbf{1}$ from *flights* to *were*, the loss at *were* reaches the write at *flights* intact. The gates depend on $\mathbf{h}_{t-1}$ too, so there are other paths; this is the one that survives.

Hochreiter & Schmidhuber (1997); Gers et al. (2000); Goodfellow et al. (2016), §10.10.1.

Why the cell remembers

Proposition 5 (The cell's gradient is a product of forget gates)

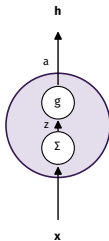
Holding the gates fixed, along the cell's own path $\partial \mathbf{c}_t / \partial \mathbf{c}_k = \prod_{s=k+1}^t \text{diag}(\mathbf{f}_s)$, a product of numbers in $(0, 1)$ with no weight matrix and no activation derivative in it: with $\mathbf{f}_s \approx \mathbf{1}$ the gradient passes unattenuated.

- Proof. $\mathbf{c}_s = \mathbf{f}_s \odot \mathbf{c}_{s-1} + \mathbf{j}_s$ is linear in $\mathbf{c}_{s-1}$ with coefficient $\mathbf{f}_s$; chain the steps. □
- Against Proposition 2: the simple cell's factor $\text{diag}(g')$ U is typically well below one; the LSTM's is a gate the network can hold at one — the constant error carousel. On the sentence: with $\mathbf{f}_s \approx \mathbf{1}$ from *flights* to *were*, the loss at *were* reaches the write at *flights* intact. The gates depend on $\mathbf{h}_{t-1}$ too, so there are other paths; this is the one that survives.

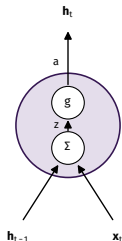
Hochreiter & Schmidhuber (1997); Gers et al. (2000); Goodfellow et al. (2016), §10.10.1.

Gated units are drop-in modules

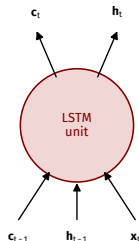
(a) a feedforward unit



(b) a simple recurrent unit



(c) an LSTM unit



The three units side by side, in the layout of Jurafsky's Fig. 14.14. (a) A feedforward unit: a weighted sum z , an activation g , an output. (b) A simple recurrent unit: the same with two inputs, h_{t-1} and x_t . (c) An LSTM unit: everything of the last two slides is inside the circle; from outside, the only change is a second vector in, c_{t-1} , and a second vector out, c_t . So an LSTM drops into every architecture of Classes 27 and 29 — encoder, decoder, stacked, bidirectional — and is unrolled and trained exactly as in Class 26.

Layout of Jurafsky & Martin (2026), Fig. 14.14, §14.5.1.

Other Gated Cells

The gated recurrent unit

Definition 6 (GRU)

One gate decides both what to forget and what to write:

$$\mathbf{u}_t = \sigma(\mathbf{U}_u \mathbf{h}_{t-1} + \mathbf{W}_u \mathbf{x}_t) \quad \text{update gate}$$

$$\mathbf{r}_t = \sigma(\mathbf{U}_r \mathbf{h}_{t-1} + \mathbf{W}_r \mathbf{x}_t) \quad \text{reset gate}$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{U}_h(\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{W}_h \mathbf{x}_t) \quad \text{candidate}$$

$$\mathbf{h}_t = \mathbf{u}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{u}_t) \odot \tilde{\mathbf{h}}_t \quad \text{new state}$$

- The same idea with less equipment: no separate notebook, three weight pairs against four. The update gate $\mathbf{u}_t$ is eraser and pen in one — what is kept of the old state is exactly what is not overwritten by the new; the reset gate chooses what of the past the candidate may see

Goodfellow et al. (2016), Eqs. 10.45–10.47; Cho et al. (2014); Chung et al. (2014).

The GRU's state stays between old and new

Proposition 7 (Convex update)

Since every entry of $\mathbf{u}_t$ lies in $(0, 1)$, each coordinate of $\mathbf{h}_t = \mathbf{u}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{u}_t) \odot \tilde{\mathbf{h}}_t$ lies between the old state and the candidate. Along the old state the derivative is $\text{diag}(\mathbf{u}_t)$, so a coordinate with $u \approx 1$ is copied unchanged and its gradient passes unchanged, exactly as on the LSTM's cell line.

- **Proof.** $u a + (1 - u) b$ with $u \in (0, 1)$ is a convex combination of a and b ; differentiate in a . $\square$
- Which pieces of the LSTM are needed? Ablations find no variant that beats both LSTM and GRU across tasks; the forget gate is the crucial ingredient, and a forget bias of 1 makes the LSTM as strong as the best variant.

Goodfellow et al. (2016), §10.10.2; Greff et al. (2017); Jozefowicz et al. (2015).

The GRU's state stays between old and new

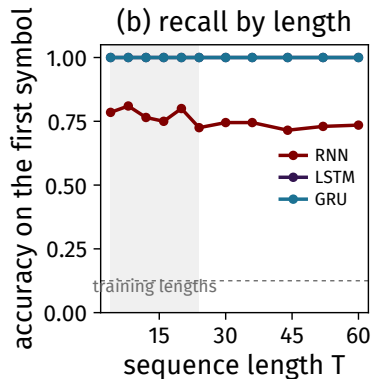
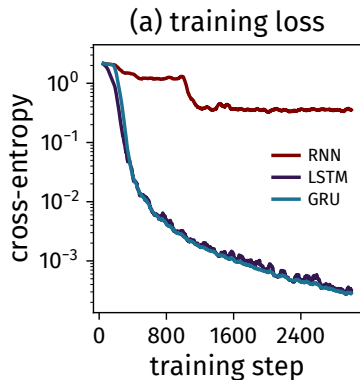
Proposition 7 (Convex update)

Since every entry of $\mathbf{u}_t$ lies in $(0, 1)$, each coordinate of $\mathbf{h}_t = \mathbf{u}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{u}_t) \odot \tilde{\mathbf{h}}_t$ lies between the old state and the candidate. Along the old state the derivative is $\text{diag}(\mathbf{u}_t)$, so a coordinate with $u \approx 1$ is copied unchanged and its gradient passes unchanged, exactly as on the LSTM's cell line.

- Proof. $u a + (1 - u) b$ with $u \in (0, 1)$ is a convex combination of a and b ; differentiate in a . $\square$
- Which pieces of the LSTM are needed? Ablations find no variant that beats both LSTM and GRU across tasks; the forget gate is the crucial ingredient, and a forget bias of 1 makes the LSTM as strong as the best variant.

Goodfellow et al. (2016), §10.10.2; Greff et al. (2017); Jozefowicz et al. (2015).

Three cells on one task



The recall task — the sentence's shape, a fact at the first step needed at the last — on an RNN, an LSTM and a GRU of width 32, trained the same way for 3,000 steps on lengths 4–24. (a) Training loss: the gated cells reach zero, the simple cell does not. (b) Accuracy by sequence length, tested to 60: the gated cells are perfect at every length, including far beyond what they trained on; the simple RNN never passes 81%, even at length 8. Chance is $1/8$.
Jurafsky & Martin (2026), §14.5; Goodfellow et al. (2016), §10.10.

Optimising Through Time

Clipping the gradient

Proposition 8 (Norm clipping keeps the direction)

Replace the gradient $\mathbf{g}$ by

$$\mathbf{g} \leftarrow \frac{\mathbf{g} v}{\|\mathbf{g}\|} \quad \text{if } \|\mathbf{g}\| > v.$$

The step is still along $\mathbf{g}$, a descent direction, and its length is at most ηv whatever the gradient's size.

- **Proof.** A positive scaling changes length only, and $\|\eta \mathbf{g} v / \|\mathbf{g}\|\| = \eta v$. □
- Why: the loss over many steps has **cliffs**, flat regions separated by walls exponentially steep in the number of steps, since $\mathbf{U}$ multiplies itself once per step; one step at a cliff can undo all the descent so far. Clipping addresses explosion only.

Goodfellow et al. (2016), §10.11.1, Eqs. 10.48–10.49; Pascanu et al. (2013); Mikolov (2012).

Clipping the gradient

Proposition 8 (Norm clipping keeps the direction)

Replace the gradient $\mathbf{g}$ by

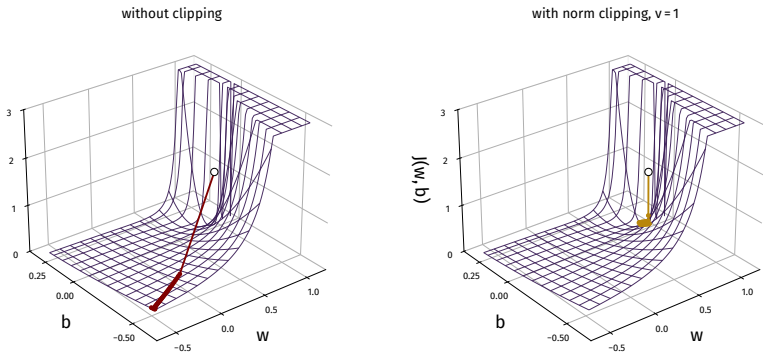
$$\mathbf{g} \leftarrow \frac{\mathbf{g} v}{\|\mathbf{g}\|} \quad \text{if } \|\mathbf{g}\| > v.$$

The step is still along $\mathbf{g}$, a descent direction, and its length is at most ηv whatever the gradient's size.

- Proof. A positive scaling changes length only, and $\|\eta \mathbf{g} v / \|\mathbf{g}\|\| = \eta v$. □
- Why: the loss over many steps has cliffs, flat regions separated by walls exponentially steep in the number of steps, since U multiplies itself once per step; one step at a cliff can undo all the descent so far. Clipping addresses explosion only.

Goodfellow et al. (2016), §10.11.1, Eqs. 10.48–10.49; Pascanu et al. (2013); Mikolov (2012).

Clipping at a cliff, drawn



The loss surface $J(w, b)$ of a recurrent net with two parameters, $h_t = w h_{t-1} + b$ for 50 steps and a squared error at the end, computed here and drawn in the layout of Goodfellow's Fig. 10.17: flat almost everywhere and exponentially steep at $w \approx 1$, because w multiplies itself once per step. Gradient descent from the same start on the lip of the cliff, with the same step size: without clipping (left) the first step at the cliff throws the parameters out of the frame, to a loss of 0.36; with the norm clipped at $v = 1$ (right) the steps are bounded and settle in the ravine at 0.005. Layout of Goodfellow et al. (2016), Fig. 10.17, after Pascanu et al. (2013).

Encouraging the gradient to flow

- Clipping handles explosion; vanishing needs a path whose product of derivatives is near one — the gates — or a **regulariser** that asks for it
- Pascanu et al. penalise, at every step, how much the back-propagated gradient shrinks in crossing that step:

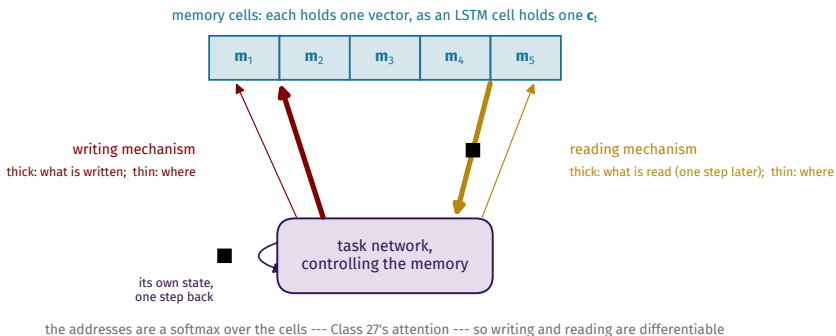
$$\Omega = \sum_t \left(\frac{\|(\nabla_{\mathbf{h}_t} L)^\top \partial \mathbf{h}_t / \partial \mathbf{h}_{t-1}\|}{\|\nabla_{\mathbf{h}_t} L\|} - 1 \right)^2,$$

treating $\nabla_{\mathbf{h}_t} L$ as a constant for the purpose of the regulariser

- With clipping alongside it, this lengthens the dependencies a simple RNN can learn; with abundant data it is not as effective as an LSTM, which is why the gate won

Goodfellow et al. (2016), §10.11.2, Eqs. 10.50–10.52; Pascanu et al. (2013).

Where the gate leads: explicit memory, drawn



A network with an explicit memory, in the layout of Goodfellow's Fig. 10.18: a row of memory cells, each holding one vector; a task network with its own recurrence (the black square is a one-step delay) that controls a writing mechanism and a reading mechanism. Thick arrows carry the content written and read; thin arrows carry the addresses the task network chooses.

Layout of Goodfellow et al. (2016), Fig. 10.18.

Where the gate leads: explicit memory, explained

- An LSTM cell is a memory of **one** vector with a learned pen, eraser and lens. Build the memory as a **set** of such cells: Weston et al.'s memory networks, Graves et al.'s neural Turing machine
- **Writing**. The task network produces a vector to store and a soft address: a softmax over the cells, so the write lands mostly in one cell and a little in the others. The pen of the LSTM, with a choice of page
- **Reading**. The same: a softmax over the cells, and the content read is the weighted average of what they hold — Class 27's attention, over memory instead of over encoder states. Because the address is a softmax and not an index, the whole thing is differentiable and trains end to end
- **Why it helps**. A fact is stored in one write and kept until overwritten; on the sentence, *flights*, *plural* goes into a cell at word two and is read back at word seven, whatever happened in between

Goodfellow et al. (2016), §10.12; Weston et al. (2014); Graves et al. (2014); Class 27.

Putting It Together

The cell, opened

	state	path across time	derivative along it
simple RNN	$\mathbf{h}_t$	through $\mathbf{U}$ and g every step	$\text{diag}(g') \mathbf{U}$, shrinks or explodes
leaky unit	μ_t	a linear self-connection	the constant α
LSTM	$\mathbf{h}_t$ and $\mathbf{c}_t$	the cell line	$\text{diag}(\mathbf{f}_t)$, chosen by the network
GRU	$\mathbf{h}_t$	the update gate's copy	$\text{diag}(\mathbf{u}_t)$, chosen by the network

Every fix is the same fix: give the gradient a path whose factors stay near one. The gate is the version where the network decides, at every step, which facts get that path.

Takeaways

1. A fact that must survive a long wait, and the gradient that would teach it, both cross the same wire once per step; the simple cell has one vector, rewritten every step, for keeping and for everything else
2. A repeated linear map raises its eigenvalues to the power of the time: components decay or explode geometrically (Theorem 1), and the same product bounds the gradient of the nonlinear cell (Proposition 2)
3. A linear self-connection of weight near one is a running average with a long time constant, and a path the gradient survives (Proposition 3)
4. The LSTM adds a copied cell and three gates — eraser, pen, lens (Definition 4); along the cell line the gradient is a product of forget gates, with no weight matrix in it (Proposition 5)
5. The GRU does the same with two gates and no separate cell; its update is a convex combination (Definition 6, Proposition 7)
6. Norm clipping bounds the step without changing its direction (Proposition 8); an LSTM is a better answer to vanishing than any regulariser

Class 29 puts the cell to work: language models, taggers, classifiers, and what stacking and bidirectionality buy.

References i

1. D. Jurafsky & J. H. Martin, *Speech and Language Processing*, 3rd ed. draft, August 2026. Ch. 14 §14.5 (Eqs. 14.20–14.27, Figs. 14.13–14.14), §14.5.1.
2. I. Goodfellow, Y. Bengio & A. Courville, *Deep Learning*, MIT Press, 2016. §10.7 (Eqs. 10.36–10.39, Fig. 10.15), §10.8, §10.9, §10.10 (Eqs. 10.40–10.47, Fig. 10.16), §10.11 (Eqs. 10.48–10.52, Fig. 10.17), §10.12 (Fig. 10.18).
3. S. Hochreiter & J. Schmidhuber, “Long short-term memory”, *Neural Computation* 9(8), 1997.
4. F. A. Gers, J. Schmidhuber & F. Cummins, “Learning to forget: continual prediction with LSTM”, *Neural Computation* 12(10), 2000.
5. Y. Bengio, P. Simard & P. Frasconi, “Learning long-term dependencies with gradient descent is difficult”, *IEEE Trans. Neural Networks* 5(2), 1994.
6. R. Pascanu, T. Mikolov & Y. Bengio, “On the difficulty of training recurrent neural networks”, ICML 2013.
7. K. Cho et al., “Learning phrase representations using RNN encoder–decoder for statistical machine translation”, EMNLP 2014; J. Chung et al., “Empirical evaluation of gated recurrent neural networks on sequence modeling”, 2014.

References ii

8. K. Greff et al., “LSTM: a search space odyssey”, *IEEE TNNLS* 28(10), 2017; R. Jozefowicz, W. Zaremba & I. Sutskever, “An empirical exploration of recurrent network architectures”, ICML 2015.
9. M. C. Mozer, “Induction of multiscale temporal structure”, NeurIPS 1992; T. Lin et al., “Learning long-term dependencies in NARX recurrent neural networks”, *IEEE Trans. Neural Networks* 7(6), 1996.
10. H. Jaeger & H. Haas, “Harnessing nonlinearity: predicting chaotic systems and saving energy in wireless communication”, *Science* 304, 2004.
11. T. Mikolov, *Statistical Language Models Based on Neural Networks*, PhD thesis, Brno University of Technology, 2012.
12. J. Weston, S. Chopra & A. Bordes, “Memory networks”, ICLR 2015 (arXiv 2014); A. Graves, G. Wayne & I. Danihelka, “Neural Turing machines”, arXiv:1410.5401, 2014.

Questions?